Assignment 2 : Simple Search Engine using Hadoop MapReduce

Ekaterina Akimenko
e.akimenko@innopolis.university

1 Introduction

The goal of this project is to build a simple search engine using distributed tools. The idea is quite straightforward. We take a large text dataset, process it step by step, and then allow users to search through it.

The system uses Hadoop MapReduce to build an inverted index. Then Spark is used to compute additional statistics. After that, everything is stored in Cassandra. Finally, users can run search queries, and results are ranked using BM25.

Even though the system is simple, it still follows the same idea as real search engines. First we index data, then store it efficiently, and finally retrieve relevant results.

2 Methodology

2.1 Dataset Preparation

The dataset consists of text documents. Each file represents one document. The naming format is important:

```
<doc_id>_<doc_title>.txt
```

This helps extract both id and title later.

At first, all documents were merged into a single file and uploaded to HDFS:

```
/input/data
```

This is needed because Hadoop works better with large combined files instead of many small ones.

2.2 Index Construction using Hadoop MapReduce

The main indexing is done with Hadoop Streaming using Python scripts.

The mapper reads documents, tokenizes text, and converts everything to lowercase. A simple regex is used for tokenization. It is not perfect, but it is fast and easy to control.

For each document, term frequency (TF) is calculated. The mapper outputs pairs grouped by terms.

The reducer collects all values for each term and builds postings lists. It also keeps document length and title.

At the end of this step, we get a raw inverted index stored in HDFS.

2.3 Post-processing with Spark

After MapReduce, Spark is used to clean and organize the data.

The output is split into several parts:

- documents
- postings
- vocabulary
- statistics

Then document frequency (DF) is computed.

Global statistics are also calculated:

- total number of documents (N)
- average document length (AVGDL)

Everything is stored in HDFS:

```
/indexer/docs
/indexer/postings
/indexer/vocab
/indexer/stats
```

2.4 Storage in Cassandra

After processing, the index is loaded into Cassandra.

The schema is split into several tables:

- documents
- vocabulary
- postings
- corpus statistics

This design keeps things simple and fast. It avoids complex joins and allows direct access to needed data.

2.5 Query Processing and Ranking

The search system uses BM25 ranking.

First, the query is tokenized the same way as documents. This is important for consistency.

Then for each term, the system retrieves:

- postings
- document lengths

- document frequency

Using these values and global statistics, BM25 scores are calculated.

If there are multiple words, scores are summed. Finally, the top 10 documents are returned.

The query script supports both command line input and standard input. This makes testing easier.

3 Demonstration

3.1 Running the System

Before running anything, services should be checked:

```
/app/start-services.sh
```

This ensures that HDFS, YARN, and Cassandra are working.

The full pipeline is executed with:

```
/app/index.sh
```

This script:

- builds the index with Hadoop
- processes it with Spark
- stores it in Cassandra

To run queries:

```
/app/search.sh <query>
```

3.2 Index Creation

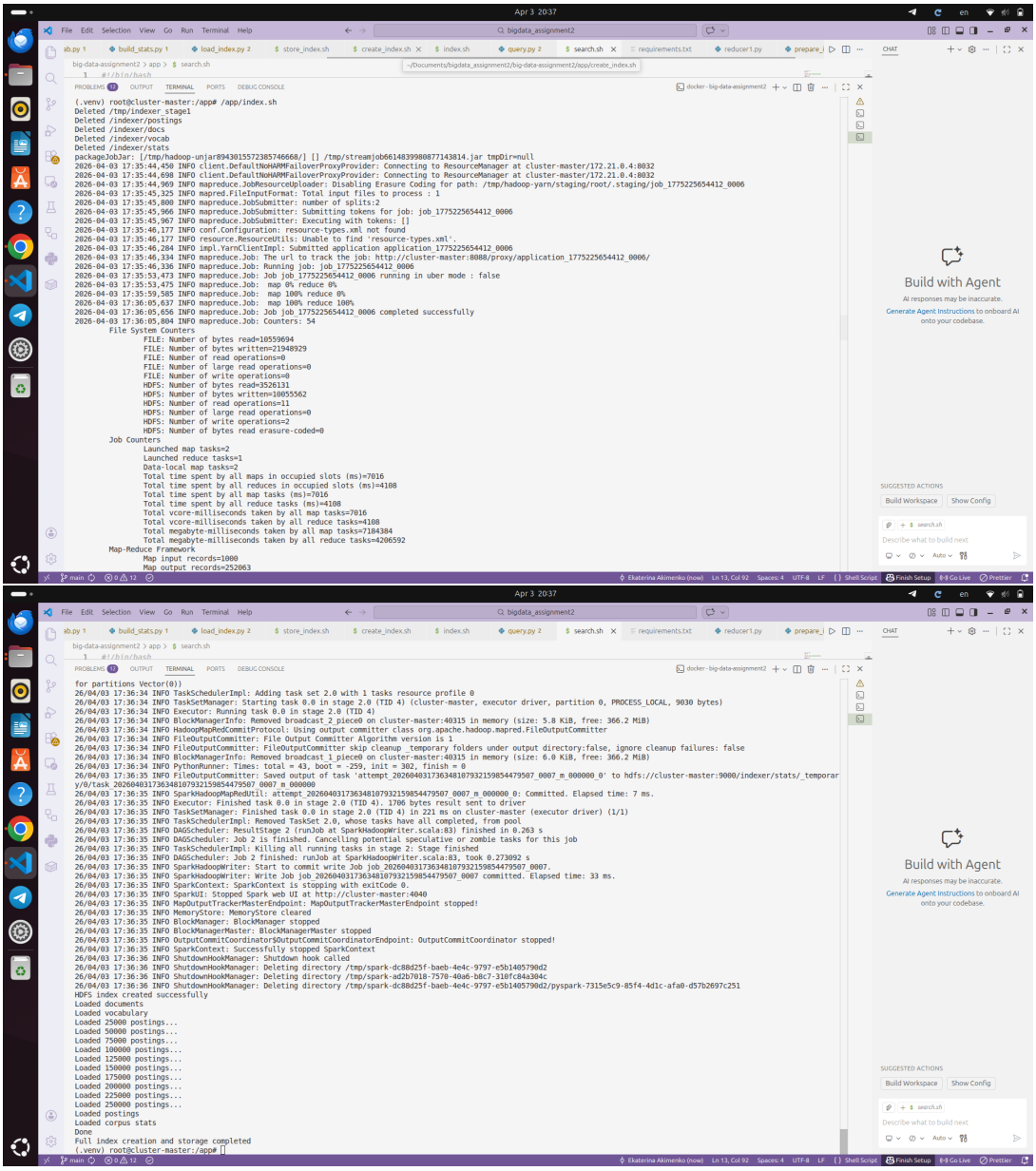

Figure 1: Successful execution of indexing pipeline

The pipeline runs step by step. Hadoop processes data first, then Spark finishes the work. It takes time, but finishes correctly.

3.3 Index Storage in HDFS

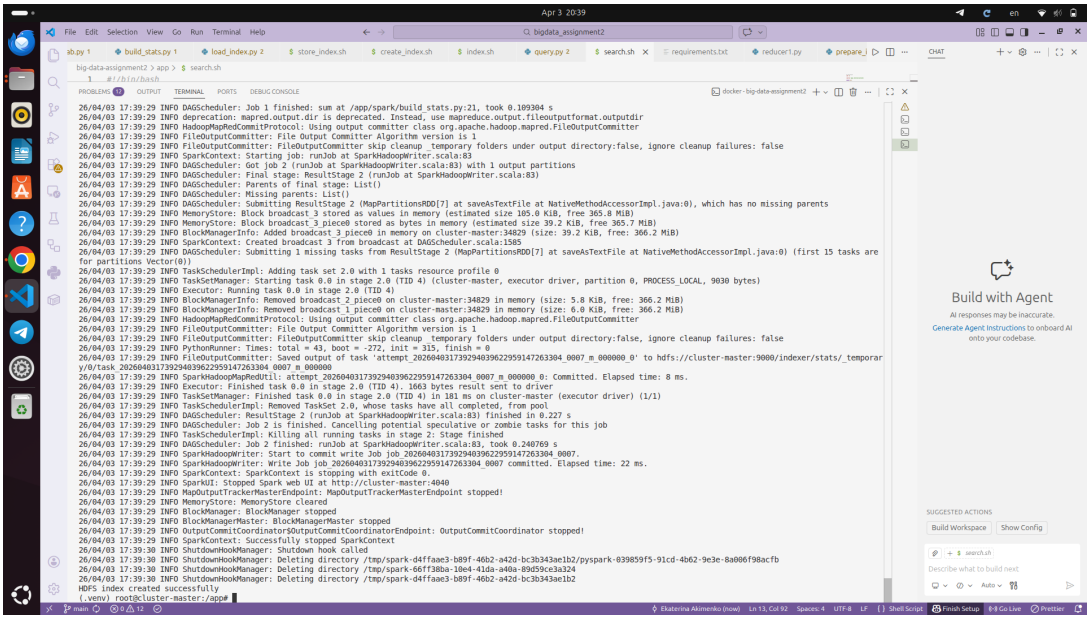

Figure 2: Index structure stored in HDFS

The index is separated into folders. This makes it easier to manage and understand.

3.4 Corpus Statistics

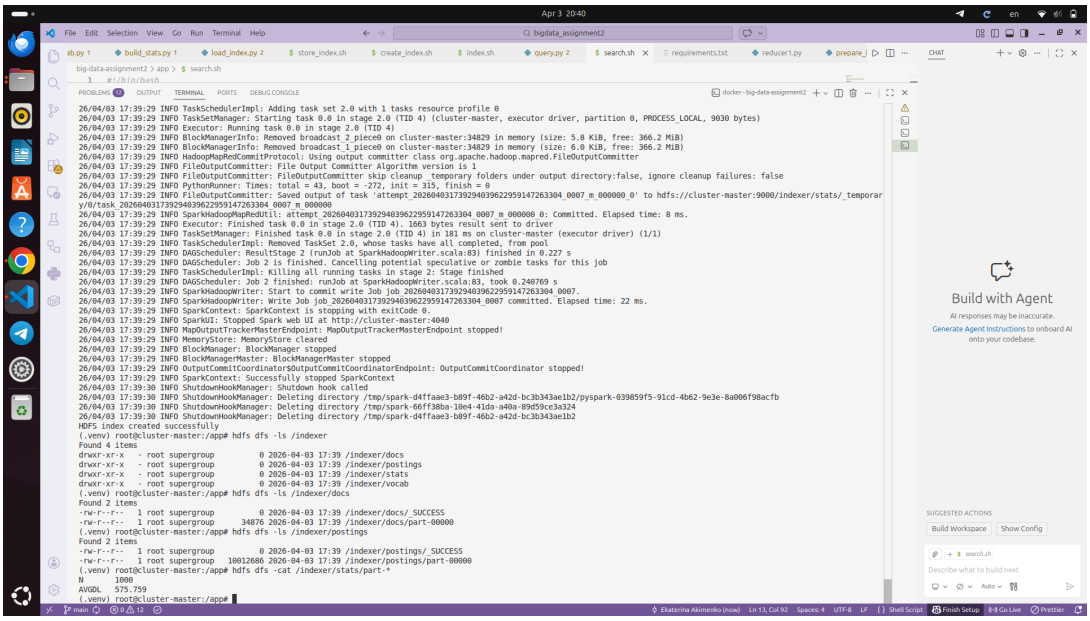

Figure 3: Corpus statistics (N and AVGDL)

These values are important for BM25 scoring.

3.5 Search Results

3.5.1 Query: history

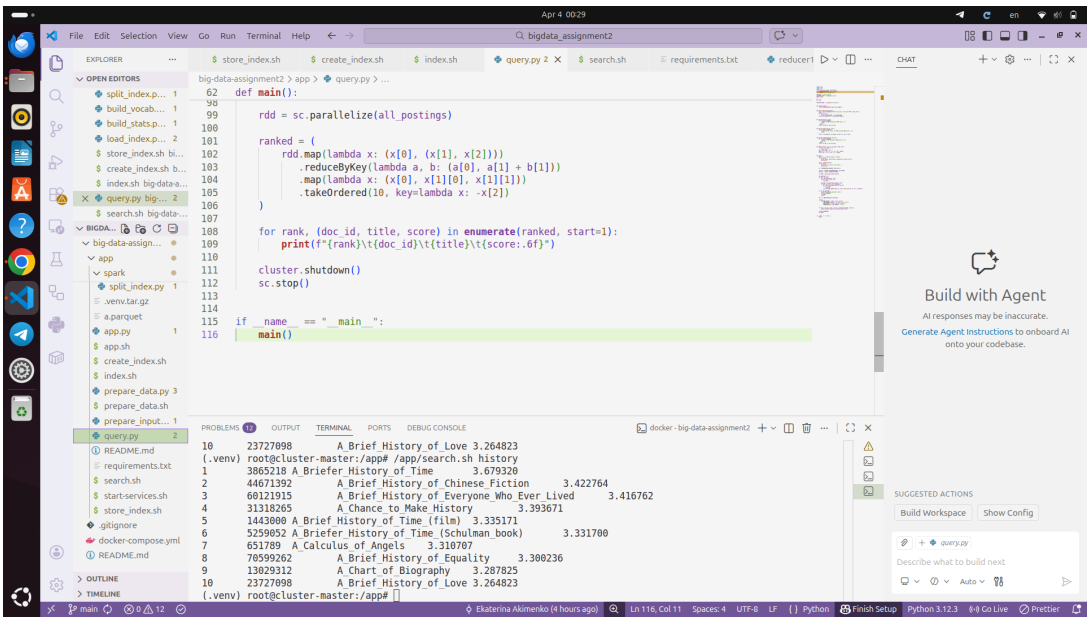

Figure 4: Search results for query: history

Most results clearly contain the word "history". This is expected.

3.5.2 Query: history of time

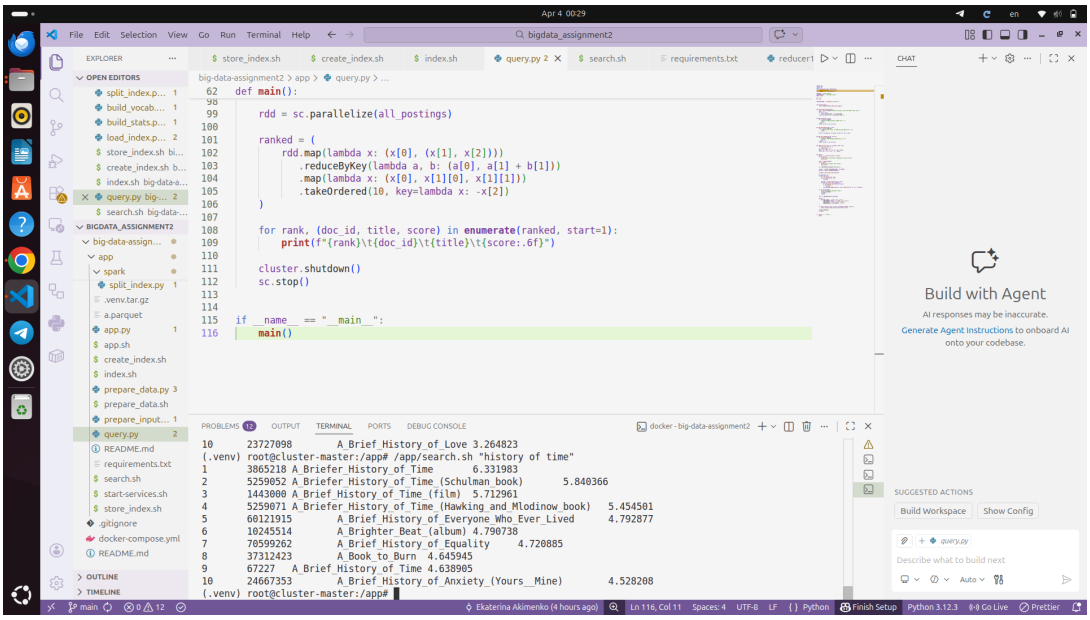

Figure 5: Search results for query: history of time

This query is more specific. The results are more precise. Several versions of "A Brief History of Time" appear.

3.5.3 Query: christmas

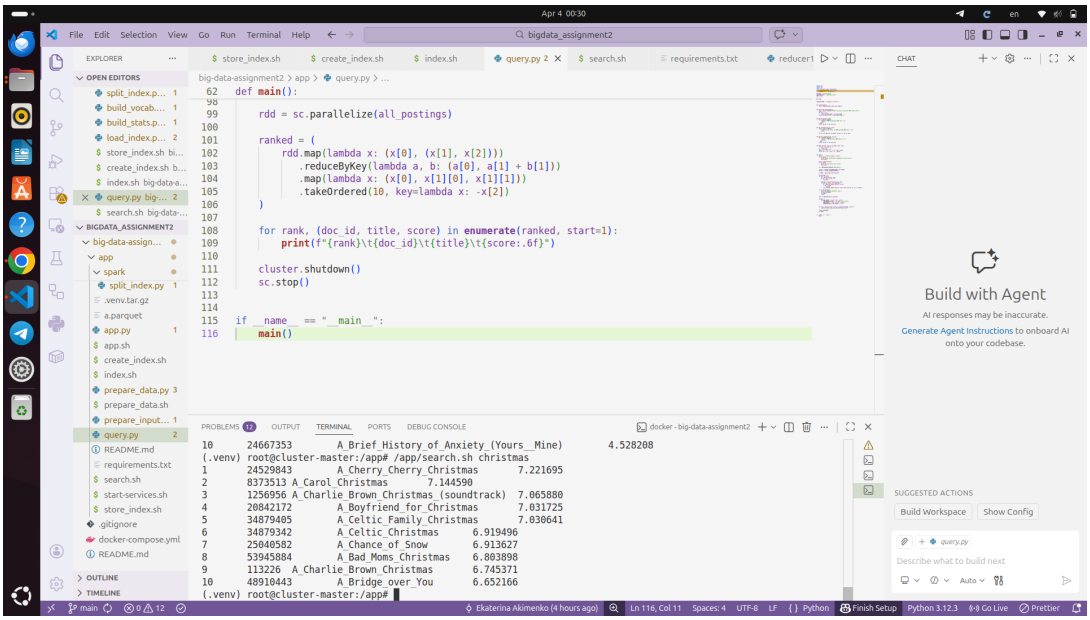

Figure 6: Search results for query: christmas

The results are clearly related to Christmas. The ranking looks correct.

3.6 Discussion

Overall, the results look reasonable. The system returns documents that are relevant to the query. More specific queries produce better results, which is expected.

BM25 seems to work well in this setup, even though the preprocessing is quite simple.

4 Results and Discussion

The system is able to retrieve meaningful results for different queries. It performs especially well when the query matches document titles.

One interesting observation is that exact matches often appear at the top. Also, combining multiple words improves ranking quality.

The tokenizer is very simple and keeps numbers as well. This is not ideal, but it keeps the system easy to implement and still works fine.

5 Conclusion

In this project, a simple search engine was implemented using Hadoop, Spark, and Cassandra.

The system builds an inverted index, computes useful statistics, stores data efficiently, and allows ranked search queries.

Even though the implementation is not very complex, it still produces good and interpretable results. This shows that even a basic approach can work quite well when combined with distributed tools.